

this feature can turn out to be highly useful. Instead of finishing one step before starting another, a developer can work on multiple parts of the project within the same interaction.

This approach saved time during experimentation, and I was able to debug logic while also asking Codex to prepare interface components that I planned to use later.

03 Long-context understanding for large codebases

One of the most common problems with AI coding tools is that they struggle when the code becomes too long. I've tried using many assistants, but each one of them loses track of context once several files are involved. This was the case with ChatGPT as well. However, the company has now improved its long-context understanding with the latest Codex models.

I tried using it on an old JavaScript project that I had not touched for quite some time. Rather than sharing my code for just one function, I decided to share a few related files for the project so that the assistant could understand the whole project better.

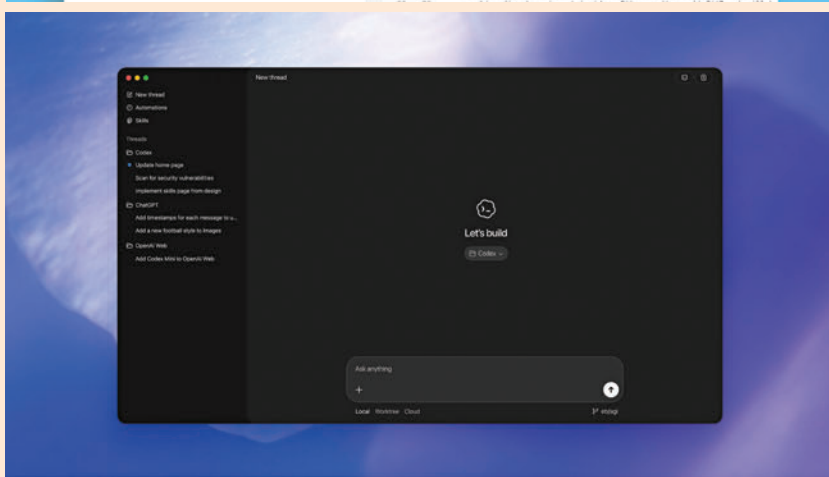
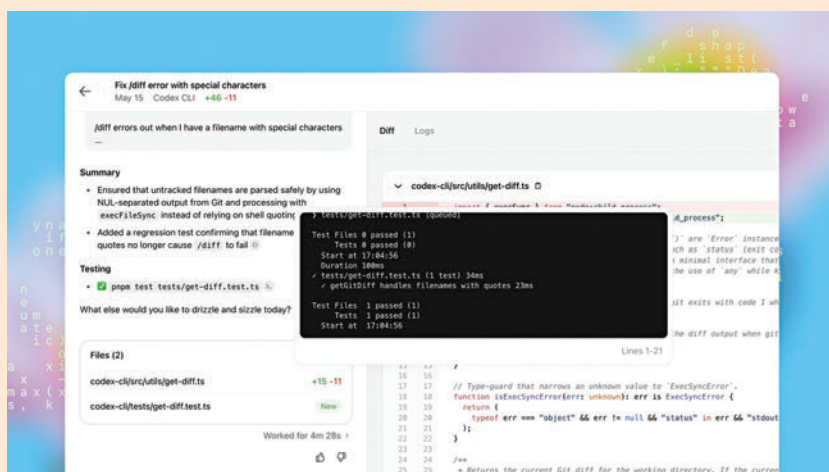
I asked the Codex assistant to review my code and suggest improvements for making it simple. The AI coding assistant provided an analysis of my code and offered several suggestions for making it simple, which was impressive.

It pointed out a function that was not being used anywhere in the project. It also pointed out some parts of the code where complicated nested conditions were used and could easily be replaced with a simple structure.

Codex did not just mark lines of code; rather, it described how the functions in the code interacted and why making certain changes would make the program easier to maintain.

04 Codebase Q&A and project understanding

OpenAI also said that Codex could now understand a whole project



and answer questions about it. So I put it to the test. Instead of asking it to write new code, I used it to help me understand some old code that I had written a few months earlier.

I pasted multiple files from the project and asked Codex a simple question about where a certain function was being used. Rather than pointing to a specific file, it actually described the connection between the function and the rest of the project.

Additionally, it described the different parts of the project that were depending on the function. Making it easier to quickly grasp the connections between different parts of the codebase without digging through every file.

handles debugging tasks when cracks appear or something in the code breaks. I gave Codex a small script with a few logical errors spread across. I then requested it to identify all the errors. Instead of simply pointing out the line of code where the errors were located, Codex actually explained the reasoning behind the errors.

However, I still wasn't satisfied, so I also ended up sharing a small test file where one of the test cases was not passing. The tool analysed the test and the function related to the test, and it then offered a small change to the function to fix the problem.

In my opinion, such debugging assistance can be particularly useful when working on larger projects where it may take a considerable amount of time to pinpoint the source of the problem for which the test is failing. [ch>](#)

05 Automatic test fixes and debugging

Lastly, I tested how Codex